The result of an arithmetic expression (involving, for example $+$, $-$, $*$ or $/$) is null if any of the input values is null. SQL treats as **unknown** the result of any comparison involving a *null* value (other than **is null** and **is not null**).

Since the predicate in a **where** clause can involve Boolean operations such as **and**, **or**, and **not** on the results of comparisons, the definitions of the Boolean operations are extended to deal with the value **unknown**, as outlined in Section 3.3.4.

- **and**: The result of *true* **and** *unknown* is *unknown*, *false* **and** *unknown* is *false*, while *unknown* **and** *unknown* is *unknown*.
- **or**: The result of *true* **or** *unknown* is *true*, *false* **or** *unknown* is *unknown*, while *unknown* **or** *unknown* is *unknown*.
- **not**: The result of **not** *unknown* is *unknown*.

SQL defines the result of an SQL statement of the form

select ... **from** $R_1, \cdots, R_n$ **where** P

to contain (projections of) tuples in $R_1 \times \cdots \times R_n$ for which predicate P evaluates to **true**. If the predicate evaluates to either **false** or **unknown** for a tuple in $R_1 \times \cdots \times R_n$ (the projection of) the tuple is not added to the result.

SQL also allows us to test whether the result of a comparison is unknown, rather than true or false, by using the clauses **is unknown** and **is not unknown**.

Null values, when they exist, also complicate the processing of aggregate operators. For example, assume that some tuples in the *loan* relation have a null value for *amount*. Consider the following query to total all loan amounts:

select sum (*amount*)
from *loan*

The values to be summed in the preceding query include null values, since some tuples have a null value for *amount*. Rather than say that the overall sum is itself *null*, the SQL standard says that the **sum** operator should ignore *null* values in its input.

In general, aggregate functions treat nulls according to the following rule: All aggregate functions except **count(*)** ignore null values in their input collection. As a result of null values being ignored, the collection of values may be empty. The **count** of an empty collection is defined to be 0, and all other aggregate operations return a value of null when applied on an empty collection. The effect of null values on some of the more complicated SQL constructs can be subtle.

A **boolean** type data, which can take values **true**, **false**, and **unknown**, was introduced in SQL:1999. The aggregate functions **some** and **every**, which mean exactly what you would intuitively expect, can be applied on a collection of Boolean values.

4.6 Nested Subqueries

SQL provides a mechanism for nesting subqueries. A subquery is a **select-from-where** expression that is nested within another query. A common use of subqueries

is to perform tests for set membership, make set comparisons, and determine set cardinality. We shall study these uses in subsequent sections.

4.6.1 Set Membership

SQL draws on the relational calculus for operations that allow testing tuples for membership in a relation. The **in** connective tests for set membership, where the set is a collection of values produced by a **select** clause. The **not in** connective tests for the absence of set membership. As an illustration, reconsider the query "Find all customers who have both a loan and an account at the bank." Earlier, we wrote such a query by intersecting two sets: the set of depositors at the bank, and the set of borrowers from the bank. We can take the alternative approach of finding all account holders at the bank who are members of the set of borrowers from the bank. Clearly, this formulation generates the same results as the previous one did, but it leads us to write our query using the **in** connective of SQL. We begin by finding all account holders, and we write the subquery

(**select** *customer-name*
from *depositor*)

We then need to find those customers who are borrowers from the bank and who appear in the list of account holders obtained in the subquery. We do so by nesting the subquery in an outer **select**. The resulting query is

select distinct *customer-name*
from *borrower*
where *customer-name* **in** (**select** *customer-name*
from *depositor*)

This example shows that it is possible to write the same query several ways in SQL. This flexibility is beneficial, since it allows a user to think about the query in the way that seems most natural. We shall see that there is a substantial amount of redundancy in SQL.

In the preceding example, we tested membership in a one-attribute relation. It is also possible to test for membership in an arbitrary relation in SQL. We can thus write the query "Find all customers who have both an account and a loan at the Perryridge branch" in yet another way:

select distinct *customer-name*
from *borrower, loan*
where *borrower.loan-number* = *loan.loan-number* **and**
branch-name = 'Perryridge' **and**
(*branch-name, customer-name*) **in**
(**select** *branch-name, customer-name*
from *depositor, account*
where *depositor.account-number* = *account.account-number*)

We use the **not in** construct in a similar way. For example, to find all customers who do have a loan at the bank, but do not have an account at the bank, we can write

```
select distinct customer-name
from borrower
where customer-name not in (select customer-name
                            from depositor)
```

The **in** and **not in** operators can also be used on enumerated sets. The following query selects the names of customers who have a loan at the bank, and whose names are neither Smith nor Jones.

```
select distinct customer-name
from borrower
where customer-name not in ('Smith', 'Jones')
```

4.6.2 Set Comparison

As an example of the ability of a nested subquery to compare sets, consider the query "Find the names of all branches that have assets greater than those of at least one branch located in Brooklyn." In Section 4.2.5, we wrote this query as follows:

```
select distinct T.branch-name
from branch as T, branch as S
where T.assets > S.assets and S.branch-city = 'Brooklyn'
```

SQL does, however, offer an alternative style for writing the preceding query. The phrase "greater than at least one" is represented in SQL by > **some**. This construct allows us to rewrite the query in a form that resembles closely our formulation of the query in English.

```
select branch-name
from branch
where assets > some (select assets
                     from branch
                     where branch-city = 'Brooklyn')
```

The subquery

```
(select assets
 from branch
 where branch-city = 'Brooklyn')
```

generates the set of all asset values for all branches in Brooklyn. The > **some** comparison in the **where** clause of the outer **select** is true if the *assets* value of the tuple is greater than at least one member of the set of all asset values for branches in Brooklyn.

SQL also allows < **some**, <= **some**, >= **some**, = **some**, and <> **some** comparisons. As an exercise, verify that = **some** is identical to **in**, whereas <> **some** is *not* the same as **not in**. The keyword **any** is synonymous to **some** in SQL. Early versions of SQL allowed only **any**. Later versions added the alternative **some** to avoid the linguistic ambiguity of the word *any* in English.

Now we modify our query slightly. Let us find the names of all branches that have an asset value greater than that of each branch in Brooklyn. The construct > **all** corresponds to the phrase "greater than all." Using this construct, we write the query as follows:

```
select branch-name
from branch
where assets > all (select assets
                    from branch
                    where branch-city = 'Brooklyn')
```

As it does for **some**, SQL also allows < **all**, <= **all**, >= **all**, = **all**, and <> **all** comparisons. As an exercise, verify that <> **all** is identical to **not in**.

As another example of set comparisons, consider the query "Find the branch that has the highest average balance." Aggregate functions cannot be composed in SQL. Thus, we cannot use **max** (**avg** (. . .)). Instead, we can follow this strategy: We begin by writing a query to find all average balances, and then nest it as a subquery of a larger query that finds those branches for which the average balance is greater than or equal to all average balances:

```
select branch-name
from account
group by branch-name
having avg (balance) >= all (select avg (balance)
                             from account
                             group by branch-name)
```

4.6.3 Test for Empty Relations

SQL includes a feature for testing whether a subquery has any tuples in its result. The **exists** construct returns the value **true** if the argument subquery is nonempty. Using the **exists** construct, we can write the query "Find all customers who have both an account and a loan at the bank" in still another way:

```
select customer-name
from borrower
where exists (select *
              from depositor
              where depositor.customer-name = borrower.customer-name)
```

We can test for the nonexistence of tuples in a subquery by using the **not exists** construct. We can use the **not exists** construct to simulate the set containment

(that is, superset) operation: We can write "relation A contains relation B" as "**not exists** (B **except** A)." (Although it is not part of the SQL-92 and SQL:1999 standards, the **contains** operator was present in some early relational systems.) To illustrate the **not exists** operator, consider again the query "Find all customers who have an account at all the branches located in Brooklyn." For each customer, we need to see whether the set of all branches at which that customer has an account contains the set of all branches in Brooklyn. Using the **except** construct, we can write the query as follows:

```
select distinct S.customer-name
from depositor as S
where not exists ((select branch-name
                   from branch
                   where branch-city = 'Brooklyn')
                  except
                   (select R.branch-name
                    from depositor as T, account as R
                    where T.account-number = R.account-number and
                          S.customer-name = T.customer-name))
```

Here, the subquery

```
(select branch-name
 from branch
 where branch-city = 'Brooklyn')
```

finds all the branches in Brooklyn. The subquery

```
(select R.branch-name
 from depositor as T, account as R
 where T.account-number = R.account-number and
       S.customer-name = T.customer-name)
```

finds all the branches at which customer *S.customer-name* has an account. Thus, the outer **select** takes each customer and tests whether the set of all branches at which that customer has an account contains the set of all branches located in Brooklyn.

In queries that contain subqueries, a scoping rule applies for tuple variables. In a subquery, according to the rule, it is legal to use only tuple variables defined in the subquery itself or in any query that contains the subquery. If a tuple variable is defined both locally in a subquery and globally in a containing query, the local definition applies. This rule is analogous to the usual scoping rules used for variables in programming languages.

4.6.4 Test for the Absence of Duplicate Tuples

SQL includes a feature for testing whether a subquery has any duplicate tuples in its result. The **unique** construct returns the value **true** if the argument subquery contains